

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)****Department of Computer Science and Engineering****ASSESSMENT TOOL 1 – DEBUGGING**

|                       |                                                                                                                                                                                                           |                      |                               |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|-------------------------------|
| <b>Course Code:</b>   | CSA0901                                                                                                                                                                                                   | <b>Course Title:</b> | <b>Programming in Java</b>    |
| <b>CO Assessed:</b>   | CO3 – Precision (BL3,BL4)                                                                                                                                                                                 | <b>Assessment:</b>   | ASSESSMENT TOOL 1 - DEBUGGING |
| <b>Weightage:</b>     |                                                                                                                                                                                                           | <b>Total Marks:</b>  | 100                           |
| <b>CO1 Statement:</b> | Build multithreaded Java programs by applying thread priorities, synchronisation, and inter-thread communication mechanisms to achieve concurrent program execution and use exception handling techniques |                      |                               |
| <b>Student Name:</b>  |                                                                                                                                                                                                           | <b>Reg. No.:</b>     |                               |
| <b>Section Batch:</b> | / SLOT B                                                                                                                                                                                                  | <b>Date:</b>         |                               |

**Code of Conduct**

I \_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: \_\_\_\_\_

**Instructions**

- This test contains 4 questions Total: 100 Marks.
- No negative marking. Un answered questions carry zero marks.
- No calculators or electronic devices permitted.

**Rubrics for Calculation**

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|----------|-----------|---------------------|----------------------|----------------------|---------------------|
|----------|-----------|---------------------|----------------------|----------------------|---------------------|

|                            |     |                                                   |                                    |                                   |                                    |
|----------------------------|-----|---------------------------------------------------|------------------------------------|-----------------------------------|------------------------------------|
| Error Identification       | 20% | Identifies all errors accurately                  | Identifies most errors             | Identifies some errors            | Unable to identify errors          |
| Java Concept Understanding | 25% | Demonstrates excellent understanding              | Demonstrates good understanding    | Demonstrates basic understanding  | Demonstrates limited understanding |
| Debugging Skills           | 20% | Uses systematic debugging effectively             | Uses appropriate debugging methods | Uses debugging with some guidance | Unable to debug independently      |
| Code Correction            | 20% | Corrects all errors and produces correct output   | Corrects most errors               | Corrects some errors              | Major errors remain                |
| Testing & Explanation      | 15% | Tests thoroughly and clearly explains corrections | Tests and explains adequately      | Limited testing/explanation       | No proper testing or explanation   |

### Q1. Exception Handling – try-catch-finally

Identify and correct the errors in the following program. Predict the output after debugging and explain the role of finally.

```

class Test {
    public static void main(String[] args) {
        try {
            int a = 10 / 0;
            System.out.println("A = " + a);
        }
        catch (NullPointerException e) {
            System.out.println("Null Error");
        }
        finally {
            System.out.println("Finally Block");
        }

        System.out.println("Program Completed");
    }
}

```

**Debugging Task:** Correct the exception handling so that the program terminates normally after displaying an appropriate error message.

### Corrected Program

```
class Test {  
    public static void main(String[] args) {  
        try {  
            int a = 10 / 0;  
            System.out.println("A = " + a);  
        }  
  
        catch (ArithmeticException e) {  
            System.out.println("Arithmetic Error: Cannot divide by  
zero");  
        }  
  
        finally {  
            System.out.println("Finally Block");  
        }  
  
        System.out.println("Program Completed");  
    }  
}
```

### Output

```
Arithmetic Error: Cannot divide by zero  
Finally Block  
Program Completed
```

---

### Q2. User-Defined Exception – Hard

Debug the following program so that it correctly throws and handles the user-defined exception when the balance is insufficient.

```
class InsufficientBalance extends Exception {  
    InsufficientBalance(String msg) {  
        super(msg);  
    }  
}  
  
class Bank {  
    static void withdraw(int balance, int amount) {  
        if (amount > balance)  
            throw new InsufficientBalance("Insufficient Balance");  
    }  
}
```

```

        balance = balance - amount;
        System.out.println("Remaining Balance: " + balance);
    }

    public static void main(String[] args) {
        try {
            withdraw(5000, 7000);
        }
        catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}

```

**Debugging Task:** Identify the compilation error, modify the method appropriately, and ensure that the custom exception is correctly propagated and handled.

### Corrected Program

```

class InsufficientBalance extends Exception {

    InsufficientBalance(String msg) {
        super(msg);
    }
}

class Bank {

    static void withdraw(int balance, int amount)
        throws InsufficientBalance {

        if (amount > balance) {
            throw new InsufficientBalance("Insufficient Balance");
        }

        balance = balance - amount;

        System.out.println("Remaining Balance: " + balance);
    }

    public static void main(String[] args) {

        try {
            withdraw(5000, 7000);
        }

        catch (InsufficientBalance e) {
            System.out.println(e.getMessage());
        }

    }
}

```

### Output

### Q3. Multithreading – Synchronization

The following program is intended to safely increment a shared counter using two threads. Debug it so that the final counter value is always **2000**.

```
class Counter {
    int count = 0;

    void increment() {
        count++;
    }
}
class MyThread extends Thread {
    Counter c;

    MyThread(Counter c) {
        this.c = c;
    }

    public void run() {
        for (int i = 0; i < 1000; i++)
            c.increment();
    }
}
class Test {
    public static void main(String[] args) throws Exception {
        Counter c = new Counter();

        Thread t1 = new MyThread(c);
        Thread t2 = new MyThread(c);

        t1.start();
        t2.start();

        System.out.println("Final Count = " + c.count);
    }
}
```

**Debugging Task:** Identify the synchronization and execution-order problems. Modify the program so that the final output is reliably:

Final Count = 2000

**Corrected Program**

```

class Counter {
    int count = 0;

    synchronized void increment() {
        count++;
    }
}
class MyThread extends Thread {
    Counter c;

    MyThread(Counter c) {
        this.c = c;
    }

    public void run() {
        for (int i = 0; i < 1000; i++) {
            c.increment();
        }
    }
}
class Test {
    public static void main(String[] args) throws Exception {
        Counter c = new Counter();
        Thread t1 = new MyThread(c);
        Thread t2 = new MyThread(c);

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("Final Count = " + c.count);
    }
}

```

## Output

```
Final Count = 2000
```

---

## Q4. Inter-Thread Communication – wait() and notify()

Debug the following producer-consumer program. The program may terminate incorrectly or remain waiting indefinitely.

```
class Shared {
```

```

int value;
boolean available = false;

synchronized void produce(int v) {
    value = v;
    available = true;
    notify();
}

synchronized void consume() {
    while (!available) {
        try {
            wait();
        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }

    System.out.println("Consumed: " + value);
    available = false;
}
}

class Test {
    public static void main(String[] args) {
        Shared s = new Shared();

        Thread consumer = new Thread(() -> s.consume());
        Thread producer = new Thread(() -> s.produce(100));

        consumer.start();
        producer.start();
    }
}

```

**Debugging Task:** Modify the program to support **multiple produce-consume operations** and ensure that wait(), notify()/notifyAll(), and synchronization are used correctly.

### Corrected Program

```

class Shared {

    int value;
    boolean available = false;

    synchronized void produce(int v) {

        while (available) {
            try {
                wait();
            } catch (InterruptedException e) {

```

```

        Thread.currentThread().interrupt();
        return;
    }
}

value = v;
available = true;

System.out.println("Produced: " + value);

notifyAll();
}

synchronized void consume() {

    while (!available) {
        try {
            wait();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return;
        }
    }

    System.out.println("Consumed: " + value);

    available = false;

    notifyAll();
}

class Test {

    public static void main(String[] args) throws Exception {

        Shared s = new Shared();

        Thread consumer = new Thread(() -> {
            for (int i = 1; i <= 5; i++) {
                s.consume();
            }
        });

        Thread producer = new Thread(() -> {
            for (int i = 1; i <= 5; i++) {
                s.produce(i * 100);
            }
        });

        consumer.start();
        producer.start();

        producer.join();
        consumer.join();
    }
}

```

```
        System.out.println("Production and consumption completed.");  
    }  
}
```

## Output

```
Produced: 100  
Consumed: 100  
Produced: 200  
Consumed: 200  
Produced: 300  
Consumed: 300  
Produced: 400  
Consumed: 400  
Produced: 500  
Consumed: 500
```

---